

Manipulación de cadenas con stringr : : GUÍA RÁPIDA

El paquete stringr proporciona un conjunto de herramientas internamente coherentes para trabajar con cadenas de caracteres, es decir, secuencias de caracteres entre comillas.



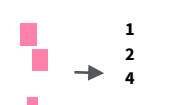
Detectar Coincidencias



str_detect(string, **pattern**, negate = FALSE) Detecte la presencia de una coincidencia de patrón en una cadena. Además, **str_like()**. `str_detect(fruit, "a")`



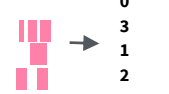
str_starts(string, **pattern**, negate = FALSE) Detecta la presencia de una coincidencia de patrón al principio de una cadena. Además, **str_ends()**. `str_starts(fruit, "a")`



str_which(string, **pattern**, negate = FALSE) Buscar los índices de las cadenas que contienen una coincidencia de patrón. `str_which(fruit, "a")`

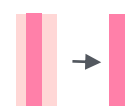


str_locate(string, **pattern**) Localice las posiciones de las coincidencias de patrones en una cadena. Además, **str_locate_all()**. `str_locate(fruit, "a")`

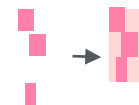


str_count(string, **pattern**) Contar el número de coincidencias en una cadena. `str_count(fruit, "a")`

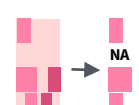
Subconjuntos de Texto



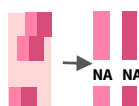
str_sub(string, start = 1L, end = -1L) Extraer subcadenas de un vector de caracteres. `str_sub(fruit, 1, 3)`; `str_sub(fruit, -2)`



str_subset(string, **pattern**, negate = FALSE) Devuelve solo las cadenas que contienen una coincidencia de patrón. `str_subset(fruit, "p")`

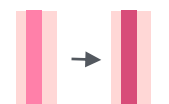


str_extract(string, **pattern**) Devuelve la primera coincidencia de patrón encontrada en cada cadena, como un vector. Además, **str_extract_all()** para devolver todas las coincidencias de patrones. `str_extract(fruit, "[aeiou]")`

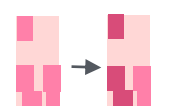


str_match(string, **pattern**) Devuelve la primera coincidencia de patrón encontrada en cada cadena, como una matriz con una columna para cada grupo () en el patrón. Además **str_match_all()**. `str_match(sentences, "(a|the) ([^+])")`

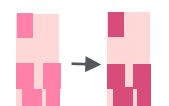
Mutar Cadenas de Texto



str_sub() <- value. Reemplace las subcadenas identificando las subcadenas con **str_sub()** y asignándolas a los resultados. `str_sub(fruit, 1, 3) <- "str"`



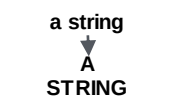
str_replace(string, **pattern**, replacement) Reemplace el primer patrón coincidente en cada cadena. Además, **str_remove()**. `str_replace(fruit, "p", "-")`



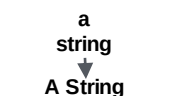
str_replace_all(string, **pattern**, replacement) Reemplace todos los patrones coincidentes en cada cadena. Además, **str_remove_all()**. `str_replace_all(fruit, "p", "-")`



str_to_lower(string, locale = "en")¹ Convierte las cadenas a minúsculas. `str_to_lower(sentences)`



str_to_upper(string, locale = "en")¹ Convierte las cadenas a mayúsculas. `str_to_upper(sentences)`

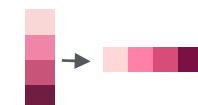


str_to_title(string, locale = "en")¹ Convierta las cadenas a mayúsculas y minúsculas. Además **str_to_sentence()**. `str_to_title(sentences)`

Unir y Dividir



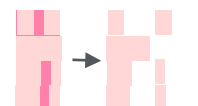
str_c(..., sep = "", collapse = NULL) Une varias cadenas en una sola cadena. `str_c(letters, LETTERS)`



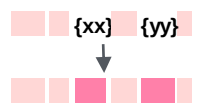
str_flatten(string, collapse = "") Se combina en una sola cadena, separada por contracción. `str_flatten(fruit, ", ")`



str_dup(string, times) Repetir cadenas veces veces. Además, **str_unique()** para eliminar duplicados. `str_dup(fruit, times = 2)`



str_split_fixed(string, **pattern**, n) Divide un vector de cadenas en una matriz de subcadenas (división en las apariciones de una coincidencia de patrón). Además, **str_split()** to devuelve una lista de subcadenas y **str_split_n()** para devolver la enésima subcadena. `str_split_fixed(sentences, " ", n=3)`

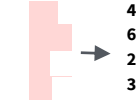


str_glue(..., .sep = "", .envir = parent.frame()) Cree una cadena a partir de cadenas y {expresiones} para evaluarla. `str_glue("Pi is {pi}")`

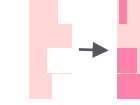


str_glue_data(x, ..., .sep = "", .envir = parent.frame(), .na = "NA") Utilice un marco de datos, una lista o un entorno para crear una cadena a partir de cadenas y {expresiones} para evaluarla. `str_glue_data(mtcars, "{rownames(mtcars)} has {hp} hp")`

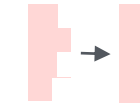
Trabajar con Longitudes



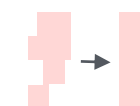
str_length(string) El ancho de los textos (es decir, el número de puntos de código, que generalmente es igual al número de caracteres). `str_length(fruit)`



str_pad(string, width, side = c("left", "right", "both"), pad = " ") Rellene los textos a un ancho constante. `str_pad(fruit, 17)`



str_trunc(string, width, side = c("right", "left", "center"), ellipsis = "...") Trunque el ancho de los textos y reemplace el contenido por puntos suspensivos. `str_trunc(sentences, 6)`



str_trim(string, side = c("both", "left", "right")) Recortar espacios en blanco desde el principio y/o el final de un texto. `str_trim(str_pad(fruit, 17))`



str_squish(string) Recorta los espacios en blanco de cada extremo y contrae varios espacios en espacios individuales. `str_squish(str_pad(fruit, 17, "both"))`

Ordenar Cadenas de Caracteres



str_order(x, decreasing = FALSE, na.last = TRUE, locale = "en", numeric = FALSE, ...) ¹ Devuelve el vector de índices que ordena un vector de caracteres. `fruit[str_order(fruit)]`
str_sort(x, decreasing = FALSE, na.last = TRUE, locale = "en", numeric = FALSE, ...) ¹ Ordenar un vector de caracteres. `str_sort(fruit)`



Ayudantes

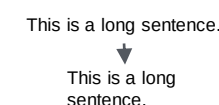


str_conv(string, encoding) Invalide la codificación de un texto. `str_conv(fruit, "ISO-8859-1")`

str_view(string, **pattern**, match = NA) Ver la representación HTML de todas las coincidencias de expresiones regulares. `str_view(sentences, "[aeiou]")`



str_equal(x, y, locale = "en", ignore_case = FALSE, ...) ¹ Determine si dos cadenas son equivalentes. `str_equal(c("a", "b"), c("a", "c"))`



str_wrap(string, width = 80, indent = 0, exdent = 0) Envuelve cadenas en párrafos con un buen formato. `str_wrap(sentences, 20)`

¹ Vea bit.ly/ISO639-1 para obtener una lista completa de las configuraciones regionales.

Lo que Necesitas Saber

Los argumentos de patrón en stringr se interpretan como expresiones regulares después de que se hayan analizado los caracteres especiales.

En R, las expresiones regulares se escriben como cadenas, secuencias de caracteres entre comillas (") o comillas simples (').

Algunos caracteres no se pueden representar directamente en una cadena de R. Estos deben representarse como caracteres especiales, secuencias de caracteres que tienen un significado específico, p. ej.

Especial	Carácter	Representa
\\	\	
\"	"	
\n		nueva línea

Ejecuta ?"" para ver una lista completa

Debido a esto, cada vez que aparece un \ en una expresión regular, debe escribirlo como \\ en la cadena que representa la expresión regular.

Usa writeLines() para ver cómo ve R tu cadena después de que se hayan analizado todos los caracteres especiales.

```
writeLines("\\.")
# \.
```

```
writeLines("\\ is a backslash")
# \ is a backslash
```

INTERPRETACIÓN

Los patrones en stringr se interpretan como expresiones regulares. Para cambiar este valor predeterminado, envuelva el patrón en una de las siguientes opciones:

regex() (pattern, ignore_case = FALSE, multiline = FALSE, comments = FALSE, dotall = FALSE, ...) Modifica una expresión regular para ignorar mayúsculas y minúsculas, hacer coincidir el final de las líneas y el final de las cadenas, permitir comentarios de R dentro de la expresión regular y/o tener . coincidir con todo, incluido \n. str_detect("i", regex("i", TRUE))

fixed() Coincide con bytes sin procesar, pero perderá algunos caracteres que se pueden representar de varias maneras (rápido). str_detect("\u0130", fixed("i"))

coll() Coincide con bytes sin formato y usará reglas de intercalación específicas de la configuración regional para reconocer caracteres que se pueden representar de varias maneras (lentas). str_detect("\u0130", coll("i", TRUE, locale = "tr"))

boundary() Coincide con los límites entre caracteres, line_breaks, oraciones o palabras. str_split(sentences, boundary("word"))



Expresiones Regulares - Las expresiones regulares, o regexp, son un lenguaje conciso para describir patrones en cadenas.

EMPAREJAR CARACTERES

texto (escriba esto)	regexp (significar esto)	empareja (que coincide con esto)	ejemplo
a (etc.)	a	a (etc.)	see("a")
.	.	.	see("\\.")
!	!	!	see("\\!")
?	?	?	see("\\?")
\\	\\	\\	see("\\\\")
(	(	(	see("\\(")
)	)	)	see("\\)")
{	{	{	see("\\{")
}	}	}	see("\\}")
\n	\n	nueva línea	see("\\n")
\t	\t	tab	see("\\t")
\s	\s	cualquier espacio en blanco (\S para espacios que no son espacios en blanco)	see("\\s")
\d	\d	cualquier dígito (\D para no dígitos)	see("\\d")
\w	\w	cualquier carácter de palabra (\W para caracteres que no son de palabra)	see("\\w")
\b	\b	Límites de palabras	see("\\b")
[:digit:]	[:digit:]	dígitos	see("[:digit:]")
[:alpha:]	[:alpha:]	letras	see("[:alpha:]")
[:lower:]	[:lower:]	letras minúsculas	see("[:lower:]")
[:upper:]	[:upper:]	letras mayúsculas	see("[:upper:]")
[:alnum:]	[:alnum:]	letras y números	see("[:alnum:]")
[:punct:]	[:punct:]	puntuación	see("[:punct:]")
[:graph:]	[:graph:]	letras, números y signos de puntuación	see("[:graph:]")
[:space:]	[:space:]	caracteres de espacio (p. ej. \s)	see("[:space:]")
[:blank:]	[:blank:]	espacio y tabulación (pero no nueva línea)	see("[:blank:]")
.	.	todos los caracteres excepto una nueva línea	see(".")

1 Muchas funciones base de R requieren que las clases se envuelvan en un segundo conjunto de [], p. ej. [[:digit:]]

ALTERNA

regexp	coincide	ejemplo
ab d	o	alt("ab d")
[abe]	uno de	alt("[abe]")
[^abe]	cualquiera menos	alt("[^abe]")
[a-c]	rango	alt("[a-c]")

ANCLAJES

regexp	coincide	ejemplo
^a	inicio de la cadena	anchor("^a")
a\$	fin de la cadena	anchor("a\$")

BUSCADORES

regexp	coinciden	ejemplos
a(=?c)	seguido de	look("a(=?c)")
a(!?c)	no seguido de	look("a(!?c)")
(?<=b)a	precedido por	look("(?<=b)a")
(?<!b)a	no precedido por	look("(?<!b)a")

CUANTIFICADORES

regexp	coincide	ejemplo
a?	cero o uno	quant("a?")
a*	cero o más	quant("a*")
a+	uno o más	quant("a+")
a{n}	exactamente n	quant("a{2}")
a{n,}	n o más	quant("a{2,}")
a{n,m}	entre n y m	quant("a{2,4}")

GRUPOS

texto	regexp	coincide	ejemplo
(escriba esto)	(significa)	(que coincide con este)	(el resultado es el mismo que ref("abba"))
\\1	\\1 (etc.)	first () group, etc.	ref("(a)(b)\\2\\1")

Utilice un número de escape para hacer referencia a los grupos de paréntesis que aparecen antes en un patrón y duplicarlos. Refiérase a cada grupo por su orden de aparición

